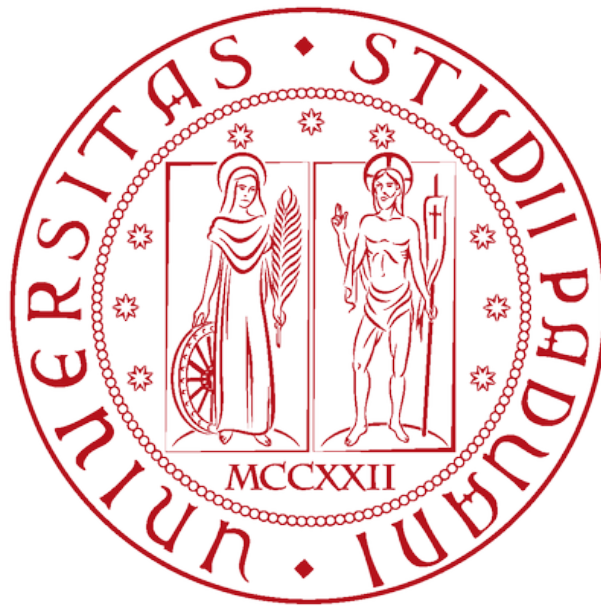


UNIVERSITY OF PADOVA

Computer Vision 2024-2025

Object Detection and Feature Matching



Middle Course Test - Group 8 (closedCV)

Bjoern Magnus Myrhaug - 2141821

Maximillian Michael Ulrich Pries - 2144162

Nihal Suri - 2141829

Academic Year 2024-2025

1 Project Description

This project focuses on developing an object detector designed to locate known objects within a set of input images depicting a scene. The detector was implemented in C++ using the OpenCV library. Given a collection of synthetic views of the target objects, the algorithm analyzes a series of test images and attempts to identify and highlight the objects. When detected, it draws a bounding box around the object and saves the coordinates in a text file. To distinguish them, each object is marked with a different color. The ideas for the algorithm were developed in group meetings with equal contributions from all team members. The individual times spent working on the project are collected in Table 1.

Table 1: Individual working hours.

Group Member	Hours Spent
Bjoern Myrhaug	14
Maximilian Pries	15
Nihal Suri	13

2 Algorithm Description

The algorithm for object detection can roughly be separated into three parts: feature detection, feature matching, and object detection.

2.1 Feature Detection

For feature detection, four types of detectors from the OpenCV library were implemented: SIFT, SURF, ORB, and AKAZE. The specific detector used during execution is selected through a choice variable. The chosen detector identifies keypoints and computes descriptors for both the training and test images. For the training images, only the descriptors are stored – organized into three separate datasets, one for each object to be detected – while the keypoints are discarded, as they are not needed for the matching process. Before the features are detected, the images can optionally be smoothed with a Gaussian filter.

2.2 Feature Matching

Once the descriptors for both the training and test images are computed, the algorithm matches each feature from the test images to the two closest training features using the nearest neighbor method provided by OpenCV’s `BFMatcher`. Having the two best matches allows to apply the ratio test: if the two matches are similarly close in feature space, the match is discarded. This helps eliminate a significant number of false positives while preserving most true positive matches. Finally, the corresponding keypoints in the test image are selected, stored, and drawn onto the image.

2.3 Object Detection

Based on the keypoint location, the bounding boxes are drawn. For this, one of three methods can be chosen:

1. **Sliding box:** Move a box of a fixed, chosen size around the image. Finds the coordinates that maximize the keypoints contained in the box. The boxes are placed both vertically and horizontally at each coordinate.
2. **Centered box:** Same as the sliding box, but once the best position is found, the box is centered around the center of mass of the corresponding keypoints.
3. **Contour based:** Generate boxes around contours found in the image. The boxes are constrained by minimum and maximum size limits. Select the box bounding the contour, that contains the highest number of keypoints.

To reduce the number of false positive detections, a threshold for a minimum number of keypoints can be chosen. If the returned box contains fewer keypoints, it is discarded, and an empty box is returned for evaluation.

3 Performance Metrics

To assess the performance of the object detection pipeline, two metrics are applied: the mean intersection over union (mIoU) and the detection accuracy.

3.1 Mean Intersection over Union

The mIoU is a popular metric used for object detection purposes. It's calculated by averaging the Intersection over Union (IoU) score for each object or class in the image or dataset. In our use case, the IoU measures the overlap of a predicted bounding box and the ground truth bounding box (whose position is provided as a text file along with the test image), divided by the total area covered by both bounding boxes. So

$$\text{IoU} = \frac{\text{area of intersection bounding boxes}}{\text{area of union bounding boxes}} \quad (1)$$

$$\text{mIoU} = \frac{\sum_{\text{all images}} \text{IoU}_{\text{image}}}{\text{number of all images}} \quad (2)$$

3.2 Detection accuracy

The detection accuracy metric is built upon the IoU calculation, and it defines if an object has been correctly identified for each category. If for one image, the predicted and ground truth bounding boxes have $\text{IoU} \geq 0.5$, the detection is considered a true-positive to exist.

4 Results

As described, the algorithm has a number of parameters to be chosen by the user. The choice of these parameters has a great impact on the performance metrics, and thus, the parameters in the final delivery have been optimized using an exhaustive search. Since the performance metrics only weakly penalize false positives, the only developer-chosen parameter is the keypoint threshold to consider the boxes. It was set from the optimized value of 3 to 20 to balance the number of false positives with the optimal performance indices. Table 2 shows the resulting parameters, and Table 3 shows the performance indices. The repository of the project can be accessed [here](#) and the resulting images can be found on the following pages.

Table 2: Parameters from optimization.

Feature Detector	SIFT
Boxing Selection	Centered Box
Sigma (Gaussian smoothing)	0.8
Cut-Off Ratio (ratio test)	0.83
Box Dimension 1	225 (pixels)
Box Dimension 2	150 (pixels)

Table 3: Resulting performance indices.

Keypoint Threshold	3 (optimized)	20 (chosen)
mIoU		
Sugar Box	0.711	0.631
Mustard Bottle	0.500	0.399
Power Drill	0.442	0.333
Overall	0.551	0.454
Accuracy		
Sugar Box	14/14	12/14
Mustard Bottle	9/14	7/14
Power Drill	7/14	5/14
Overall	30/42	24/42

